RAG Systems and Vector Databases

A concise technical reference — sample document for RAG ingestion testing

1. What is Retrieval-Augmented Generation?

Retrieval-Augmented Generation (RAG) is an architecture that combines a large language model (LLM) with an external knowledge source. Instead of relying only on knowledge frozen into the model's weights during training, a RAG system retrieves relevant documents at query time and passes them to the model as context. This grounds the model's answers in up-to-date, domain-specific data, reduces hallucinations, and allows the knowledge base to be updated without retraining the model. A typical RAG application answers questions over private data — company wikis, product documentation, support tickets, legal contracts — that no foundation model has ever seen.

2. The RAG Pipeline

- **Ingestion / Loading:** Source documents (PDFs, web pages, databases, APIs) are loaded and parsed into raw text.
- **Chunking:** Text is split into smaller passages (commonly 200–1000 tokens) so retrieval returns focused units. Overlapping chunks preserve context across boundaries.
- **Embedding:** Each chunk is converted into a dense numeric vector by an embedding model, capturing its semantic meaning.
- **Indexing:** Vectors are stored in a vector database for fast similarity search.
- **Retrieval:** The user query is embedded and the top-k most similar chunks are fetched, usually via cosine similarity or approximate nearest-neighbor (ANN) search.
- **Augmentation:** Retrieved chunks are inserted into the LLM prompt alongside the user's question.
- **Generation:** The LLM produces an answer grounded in the retrieved context, ideally citing the source chunks.

3. Chunking Guidelines

- Respect document structure: split on headings, paragraphs, or sentences rather than arbitrary character counts.
- Typical chunk sizes: 256–512 tokens for precise Q&A; 1000+ tokens for summarization-heavy tasks.
- Use 10–20% overlap between adjacent chunks to avoid cutting ideas in half.
- Store metadata (source file, page number, section title) with every chunk to enable filtering and citation.
- Re-chunk and re-embed when the embedding model changes — vectors from different models are not comparable.

4. Retrieval Strategies

Dense retrieval uses embedding similarity and excels at semantic matching — it finds “car” when you search “automobile.” Sparse retrieval (BM25, TF-IDF) matches exact keywords and performs well for names, error codes, and rare terms. Hybrid retrieval runs both and merges the ranked lists, often with Reciprocal Rank Fusion (RRF), and typically outperforms either alone. A reranker — a cross-encoder that scores each query-document pair directly — is then applied to the top 20–100 candidates to sharpen precision before the context is sent to the LLM.

5. What is a Vector Database?

A vector database stores high-dimensional embedding vectors and answers the question: “which stored vectors are closest to this query vector?” Unlike a relational database that matches exact values, a vector database performs similarity search. Because comparing a query against millions of vectors one-by-one (brute force) is too slow, production systems build an approximate nearest-neighbor index that trades a small amount of recall for orders-of-magnitude faster search.

5.1 Similarity Metrics

Metric	Formula intuition	When to use
Cosine similarity	Angle between vectors, ignores magnitude	Default for text embeddings; most models are tuned for it
Dot product	Angle and magnitude together	When embeddings are trained with dot-product objectives
Euclidean (L2)	Straight-line distance	Image embeddings, clustering, normalized vectors

5.2 Index Types

Index	How it works	Trade-off
Flat (brute force)	Compares query to every vector	Exact results; too slow beyond ~1M vectors
HNSW	Multi-layer graph of neighbors, greedy traversal	Fast and accurate; high memory use; the most common
IVF	Clusters vectors; searches only nearby clusters	Lower memory; recall depends on clusters probed
PQ / Scalar quantization	Compresses vectors into compact codes	Huge memory savings; slight accuracy loss; often combi

5.3 Popular Vector Databases

System	Type	Notable traits
FAISS	Library (local)	Fast, battle-tested; no server, persistence is manual
Chroma	Embedded / server	Simple developer experience; great for prototypes
Qdrant	Server (Rust)	Rich payload filtering, quantization, hybrid search
Pinecone	Managed cloud	Serverless scaling, zero ops; proprietary
Weaviate	Server	Built-in vectorizers, GraphQL API, hybrid search
Milvus	Distributed server	Billion-scale workloads, multiple index types
pgvector	Postgres extension	Vectors next to relational data; one database to run

6. Choosing a Vector Database

- **Under ~100k vectors:** almost anything works — pgvector or Chroma keeps the stack simple.
- **Already on Postgres:** pgvector avoids a new service and lets you join vectors with business data.
- **Heavy metadata filtering:** Qdrant and Weaviate filter during ANN search rather than after it, preserving recall.
- **No ops team:** a managed service like Pinecone removes index tuning and scaling work.
- **Billions of vectors:** distributed systems like Milvus are built for horizontal scale.

7. Evaluation Metrics

Metric	What it measures
Context Precision	How many retrieved chunks are actually relevant
Context Recall	Whether all needed information was retrieved
Faithfulness	Whether the answer is supported by the retrieved context
Answer Relevance	How well the answer addresses the user's question
Recall@k (index)	Fraction of true neighbors the ANN index returns
Latency	End-to-end response time of the pipeline

8. Common Failure Modes

- **Retrieval miss:** the right chunk exists but is not in the top-k. Fix with hybrid search, better chunking, or query rewriting.
- **Context overflow:** too many or too-long chunks exceed the model's context window. Fix with reranking and a tighter top-k.
- **Stale index:** source data changed but embeddings were not refreshed. Fix with incremental re-indexing.
- **Embedding mismatch:** query and documents embedded with different models or versions. Keep one model per index.
- **Filter-then-search recall loss:** post-filtering ANN results can leave fewer than k matches. Prefer databases that filter during search.
- **Hallucination despite context:** the model ignores retrieved text. Fix with explicit grounding instructions and citation requirements.

9. Advanced Patterns

Production systems extend the basic pipeline in several directions. **Query rewriting** reformulates vague questions before retrieval. **Multi-query retrieval** generates several query variants and merges the results. **Parent-document retrieval** searches small chunks but returns their larger parent sections, balancing precision with context. **Agentic RAG** lets the model decide when and what to

retrieve across multiple reasoning steps. **GraphRAG** builds a knowledge graph over the corpus to answer questions that require connecting facts across many documents. Finally, **semantic caching** stores embeddings of past queries so repeated questions skip the full pipeline entirely.

10. Quick Reference Checklist

- Pick an embedding model and record its name and dimension in index metadata.
- Chunk by structure with overlap; attach source metadata to every chunk.
- Start with hybrid retrieval plus a reranker; tune top-k on a labeled eval set.
- Measure context precision/recall and faithfulness before optimizing latency.
- Plan for re-indexing: data changes, and embedding models improve.